Stack Data Structure in Python

A Complete Modern Chapter for Deep Understanding

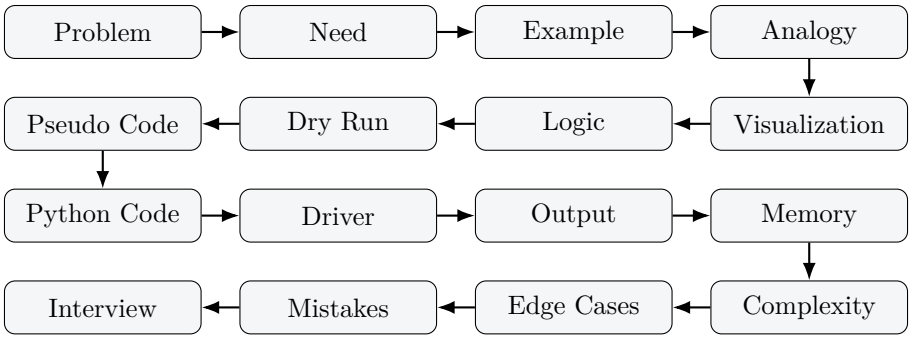
Prism Learning Notes

Contents

1 Stack Data Structure in Python

1.1 How to Study This Chapter

This chapter is designed as a complete learning experience, not a memorization sheet. For every important idea and every stack method, we will move through the same classroom sequence:



1.2 Part 1: Why Stack Exists

Problem

Imagine a pile of plates in a cafeteria. You place a clean plate on top. A customer also takes a plate from the top. Now ask yourself: **what happens if you want to remove the bottom plate?** You must first move every plate above it. That is slow, awkward, and unsafe.

Need

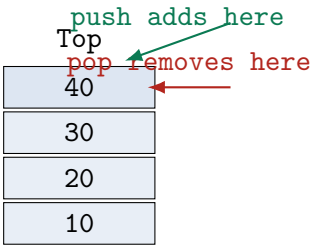
Many real problems naturally restrict access to one end only. A data structure should match the rule of the problem. If the latest item is the first item we need again, a stack is the natural structure.

Analogy

A stack is like plates, books, browser history, undo actions, recursive function calls, a narrow parking garage, a railway coach yard, and a text editor undo list. In all of them, the last thing added is usually the easiest thing to remove first.

1.2.1 Discovery Through Real Situations

Situation	What is added last?	What is removed or used first?
Stack of plates	Top plate	Top plate
Books on a desk	Last book placed	First book picked up
Undo button	Latest edit	Latest edit is undone first
Browser Back	Current page history entry	Most recent page before current
Function calls	Latest unfinished function	Latest function returns first
Expression evaluation	Latest operator or operand context	Latest pending context
Parking garage lane	Last car entering a narrow lane	First car that can leave
Reverse operations	Latest character/item scanned	First item output in reverse



1.2.2 The Discovery

Do not begin with the definition. Begin with the behavior:

1. The newest plate is at the top.
2. The top is easiest to access.
3. Removing the bottom requires disturbing everything above it.
4. Therefore, the natural rule is: **Last item added should be the first item removed.**

That rule is called **Last In, First Out**, abbreviated **LIFO**.

1.3 Part 2: What Is a Stack?

Problem

We need a structure that keeps items in order but allows insertion and deletion only at one special end.

Need

This avoids accidental access to the wrong item and makes the latest item extremely fast to retrieve.

Analogy

If a queue is a line at a ticket counter, a stack is a pile of plates. The queue serves the oldest item first. The stack serves the newest item first.

1.3.1 Definition

A **stack** is a linear data structure where insertion and deletion happen at the same end, called the **top**. It follows **LIFO**: the last item inserted is the first item removed.

1.3.2 Core Vocabulary

Term	Meaning
Top	The only active end of the stack. Push, pop, and peek use this end.
Push	Insert a new item on top.
Pop	Remove and return the top item.
Peek	Return the top item without removing it.
Size	Number of items currently inside the stack.
isEmpty	Checks whether the stack has no elements.
isFull	Checks whether a fixed-capacity stack has no free space. Python list stacks are conceptually not fixed-size.
Underflow	Trying to pop or peek from an empty stack.
Overflow	Trying to push into a full fixed-size stack.

1.3.3 Visualization

```
Top
 40   <- newest item, first to leave
 30
 20
 10   <- oldest item, last to leave
```

```
Push 50:
Before:        After:
Top            Top
 40             50
 30             40
 20             30
 10             20
                10
```

```
Pop:
Before:        Removed: 50        After:
Top                               Top
 50                                40
 40                                30
 30                                20
 20                                10
 10
```

1.4 Part 3: Stack Operations Overview

Operation	Purpose	Moves Top?	Usual Time
push	Add item at top	Yes, upward/forward	$O(1)$

pop	Remove top item	Yes, downward/backward	$O(1)$
peek	Read top item	No	$O(1)$
isEmpty	Check if no item exists	No	$O(1)$
isFull	Check fixed capacity	No	$O(1)$
size	Count items	No	$O(1)$ if stored or list length
display	Show all items top to bottom	No	$O(n)$

1.5 Part 4: Stack Using Python List

1.5.1 Why Python List Works

Problem

We need a simple storage area where the last item can be inserted and removed quickly.

Need

Python lists already provide fast operations at the right end: `append()` and `pop()`.

Analogy

Think of the right end of the list as the top of the plate stack. The list grows to the right, but when displayed as a stack, the rightmost item is shown at the top.

Python's list is implemented as a dynamic array conceptually: elements are stored in indexed slots, and Python manages capacity growth internally. We use the end of the list as top because `append(x)` and `pop()` at the end are efficient. Avoid `insert(0, x)` and `pop(0)` because they shift many elements, making them $O(n)$.

Warning

Rule: For a list-backed stack, the top is `self.items[-1]`, not `self.items[0]`.

1.5.2 Complete List-Based Stack Class

```
class ListStack:
    def __init__(self):
        self.items = []

    def push(self, value):
        self.items.append(value)

    def pop(self):
        if self.is_empty():
            raise IndexError("pop from empty stack")
        return self.items.pop()

    def peek(self):
        if self.is_empty():
            raise IndexError("peek from empty stack")
        return self.items[-1]
```

```
17
18      def is_empty(self):
19          return len(self.items) == 0
20
21      def size(self):
22          return len(self.items)
23
24      def display(self):
25          print("Top")
26          for value in reversed(self.items):
27              print(value)
28
29      def __len__(self):
30          return self.size()
31
32      def __str__(self):
33          return "Top -> " + " -> ".join(str(x) for x in reversed(self.
                items))
```

1.5.3 Method Deep Dive: `__init__`

Problem

A stack needs a private storage container before it can hold values.

Need

Initialization creates an empty stack with no top item yet.

Analogy

Before stacking plates, you need an empty table area.

Visualization:

```
items = []
Top does not exist yet.
```

Logic: Store an empty Python list in the object. **Dry Run:**

Step	Code	State
1	ListStack()	object allocated
2	self.items=[]	[]

Pseudo Code:

```
CREATE empty list
STORE it as stack storage
```

Python Code and Driver:

```
1 stack = ListStack()
2 print(stack.items)
3 print(stack.is_empty())
```

Console Output:

```
[]
True
```

Output Explanation: The list is empty, so `is_empty()` returns `True`. **Memory Before:** no stack object. **Memory After:** `stack -> ListStack object -> items -> []`. **Complexity:** Time $O(1)$, space $O(1)$ initially. **Edge Cases:** None; construction should always produce a valid empty stack. **Common Mistake:** Writing `items=[]` as a class variable; then all stacks may share storage.

Interview Discussion

A good constructor establishes the invariant: every stack object owns its own storage.

1.5.4 Method Deep Dive: `push`

Problem

We need to add a new item without disturbing older items.

Need

Push records the newest item so it becomes the next item removed.

Analogy

Place a new plate on top of the pile.

Visualization:

```
Before push(40): [10, 20, 30]     Top = 30
After  push(40): [10, 20, 30, 40] Top = 40
```

Logic: Append the value at the right end of the list. The top moves to the new last index.
Dry Run:

Step	Operation	List	Top
1	start	[10,20,30]	30
2	append(40)	[10,20,30,40]	40

Pseudo Code:

```
PUSH(value):
    add value at end of list
```

Python Code and Driver:

```
stack = ListStack()
stack.push(10)
stack.push(20)
stack.push(30)
stack.display()
```

Console Output:

```
Top
30
20
10
```

Output Explanation: `10` entered first, `20` entered second, and `30` entered last. Since stacks are LIFO, `30` is printed first as top. **Memory Before:**

```
Index:  0   1   2
Value: 10  20  30
Top index = 2
```

Memory After `push(40)`:

```
Index:  0   1   2   3
Value: 10  20  30  40
Top index = 3
```

Complexity: Amortized $O(1)$ time; $O(1)$ extra space per pushed element. Occasional internal resizing may copy elements, but averaged over many pushes it remains $O(1)$. **Edge Cases:** Pushing `None`, duplicate values, strings, or objects is allowed. **Common Mistake:** Using `insert(0, value)`; it shifts existing values and changes the top convention.

Interview Discussion

Push is $O(1)$ because it writes at the current end instead of moving all elements.

1.5.5 Method Deep Dive: `pop`

Problem

We need to remove the latest item and reveal the previous item as top.

Need

Pop is how a stack reverses recent actions: undo the newest edit, return from the newest function call, or leave the top plate.

Analogy

Take the top plate off the pile. The plate below it becomes the new top.

Visualization:

```
Before pop: [10, 20, 30] Top = 30
Removed: 30
After pop:  [10, 20]     Top = 20
```

Logic: First check underflow. Then remove and return the last list element. **Dry Run:**

Step	Check/Action	List	Return
1	is empty? no	[10,20,30]	–
2	pop last	[10,20]	30

Pseudo Code:

```
POP():
    if stack is empty: error
    remove and return last item
```

Python Code and Driver:

```
stack = ListStack()
stack.push(10)
stack.push(20)
stack.push(30)
print(stack.pop())
stack.display()
```

Console Output:

```
30
Top
20
10
```

Output Explanation: `30` was last pushed, so it is removed and printed. The remaining stack has `20` on top of `10`. **Memory Before:**

```
Index:  0   1   2
Value: 10  20  30
Top index = 2
```

Memory After:

```
Index:  0   1
Value: 10  20
Top index = 1
```

Complexity: $O(1)$ time, $O(1)$ extra space. **Edge Cases:** Empty stack causes underflow. One-item stack becomes empty after pop. **Common Mistake:** Returning `self.items[-1]` without removing it; that is peek, not pop.

Interview Discussion

Professionals check underflow before mutation and raise a clear exception such as `IndexError`.

1.5.6 Method Deep Dive: `peek`

Problem

Sometimes we need to inspect the next removable item without changing the stack.

Need

Peek supports decisions: check the current page, latest operator, or current open function frame without deleting it.

Analogy

Look at the top plate without taking it away.

Visualization:

```
Before peek: [10, 20, 30]
Returned: 30
After peek:  [10, 20, 30] unchanged
```

Logic: Check empty, then return `items[-1]`. **Dry Run:**

Step	Action	List	Return
1	is empty? no	[10,20,30]	–
2	read last	[10,20,30]	30

Pseudo Code:

```
PEEK():
    if stack is empty: error
    return last item without removing
```

Python Code and Driver:

```
stack = ListStack()
stack.push("A")
stack.push("B")
print(stack.peek())
print(stack.size())
```

Console Output:

```
B
2
```

Output Explanation: `B` is newest, so peek returns it. Size remains `2` because nothing was removed. **Memory Before and After:**

```
Index:  0   1
Value: A   B
Top index = 1
No memory link or index changes.
```

Complexity: $O(1)$ time, $O(1)$ space. **Edge Cases:** Empty stack should raise an error or return a documented sentinel. **Common Mistake:** Accidentally using `pop()` inside `peek()`.

Interview Discussion

Peek is read-only. If the size changes, the method is wrong.

1.5.7 Method Deep Dive: `is_empty`

Problem

Pop and peek become invalid when no top item exists.

Need

Empty checks prevent underflow and make client code safer.

Analogy

Before taking a plate, check whether the pile has any plates.

Visualization: `[]` means empty; `[10]` means not empty. **Logic:** Compare length with zero. **Dry Run:**

List	len	Result
`[]`	0	True
`[10]`	1	False

Pseudo Code:

```
IS_EMPTY():
    return size equals 0
```

Python Code and Driver:

```
stack = ListStack()
print(stack.is_empty())
stack.push(99)
print(stack.is_empty())
```

Console Output:

```
True
False
```

Output Explanation: The newly created stack has no values. After pushing `99`, it is no longer empty. **Memory Before/After:** Before push: `[]`. After push: `[99]`. **Complexity:** $O(1)$. **Edge Cases:** Works for all valid stacks. **Common Mistake:** Comparing list to `None`; an empty list is not `None`.

Interview Discussion

`is_empty` is small, but it protects the correctness of `pop` and `peek`.

1.5.8 Method Deep Dive: `size` and `__len__`

Problem

Users often need to know how many pending items exist.

Need

Size answers questions such as: how many undo actions remain? how deep is the call stack?

Analogy

Count how many plates are in the pile.

Logic: Return `len(self.items)`; `__len__` lets Python's built-in `len(stack)` work. **Dry Run:**

List	Operation	Result
`[10,20,30]`	size()	3
`[10,20,30]`	len(stack)	3

Pseudo Code:

```
SIZE(): return length of list
__LEN__(): return SIZE()
```

Python Code and Driver:

```
stack = ListStack()
stack.push(10)
stack.push(20)
```

```
print(stack.size())
print(len(stack))
```

Console Output:

```
2
2
```

Output Explanation: Two values were pushed, so both custom and Pythonic length operations return `2`. **Memory Before/After:** Reading size does not change `[10,20]`. **Complexity:** $O(1)$ because Python stores list length. **Edge Cases:** Empty stack size is `0`. **Common Mistake:** Manually counting with a loop for a list-backed stack; unnecessary.

Interview Discussion

`__len__` is not required by stack theory, but it makes the class feel natural in Python.

1.5.9 Method Deep Dive: `display` and `__str__`

Problem

Internal list order is bottom-to-top, but learners need to see stack order top-to-bottom.

Need

Display makes debugging and learning easier.

Analogy

When looking at a pile of plates, your eyes see the top plate first.

Logic: Traverse `reversed(self.items)`. **Dry Run:**

List	Reversed visit 1	2	3
`[10,20,30]`	30	20	10

Pseudo Code:

```
DISPLAY():
    print Top
    for each item from last to first:
        print item
```

Python Code and Driver:

```
stack = ListStack()
for value in [10, 20, 30]:
    stack.push(value)
stack.display()
print(str(stack))
```

Console Output:

```
Top
30
20
10
Top -> 30 -> 20 -> 10
```

Output Explanation: The most recent value, `30`, appears first because display intentionally shows the top first. **Memory Before/After:** Display reads `[10,20,30]` but does not mutate it. **Complexity:** $O(n)$ time, $O(1)$ extra space for printing; string creation uses $O(n)$ output space. **Edge Cases:** Empty display may print only `Top`; `__str__` could be customized to return `Empty Stack`. **Common Mistake:** Printing list directly as `[10,20,30]` and calling `10` the top.

Interview Discussion

Display is not a core abstract stack operation; it is a debugging convenience.

1.6 Part 5: Memory Diagrams for List Stack

1.6.1 Dynamic Growth Conceptually

```
Before Push:
Index:  0   1   2
Value: 10  20  30
Capacity may be larger internally, but logical size is 3.

After Push 40:
Index:  0   1   2   3
Value: 10  20  30  40
Logical size is 4. Python manages capacity growth for you.
```

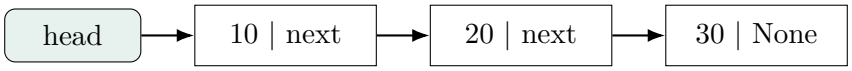
Expert Tip

When explaining memory in interviews, distinguish **logical size** from **internal capacity**. A Python list may reserve more room than currently used, but your stack only sees actual elements.

1.7 Part 6: Stack Using Singly Linked List

1.7.1 Brief Revision: Node, Head, Next

A singly linked list is made of nodes. Each node stores data and a reference to the next node. The **head** points to the first node. Traversal means following `next` links.



1.7.2 Why Head Becomes Top

Problem

If we push at the tail of a singly linked list, we may need traversal to find the last node. That is $O(n)$ unless we keep a tail pointer.

Need

A stack needs push and pop to be fast. The head is immediately accessible, so it is the perfect top.

Analogy

The head is like the visible top plate. You can place a new node before it or remove it instantly.

1.7.3 Complete Linked-List Stack Class

```python
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None

class LinkedStack:
    def __init__(self):
        self.head = None
        self.count = 0

    def push(self, value):
        new_node = Node(value)
        new_node.next = self.head
        self.head = new_node
        self.count += 1

    def pop(self):
        if self.is_empty():
            raise IndexError("pop from empty stack")
        removed_value = self.head.value
        self.head = self.head.next
        self.count -= 1
        return removed_value

    def peek(self):
        if self.is_empty():
            raise IndexError("peek from empty stack")
        return self.head.value

    def is_empty(self):
        return self.head is None

    def size(self):
        return self.count

    def display(self):
        print("Top")
        current = self.head
        while current is not None:
            print(current.value)
            current = current.next

    def __len__(self):
        return self.size()

    def __str__(self):
        values = []
        current = self.head
        while current is not None:
            values.append(str(current.value))
```

```
            current = current.next
        return "Top -> " + " -> ".join(values)
```

1.7.4 Linked Method: `Node.__init__`

Problem

Linked storage needs individual containers that can point to each other.

Need

A node packages a value with a next reference.

Analogy

A train coach stores passengers and also has a connector to the next coach.

Visualization: value: 10 next: None **Logic:** Store value; initialize next as `None`. **Dry Run:**

Step	Code	Result
1	Node(10)	value=10
2	next=None	isolated node

Pseudo Code:

```
NODE(value):
    store value
    set next to None
```

Python Code and Driver:

```
node = Node(10)
print(node.value)
print(node.next)
```

Console Output:

```
10
None
```

Output Explanation: The node stores `10` and points nowhere yet. **Memory Before/After:** Before: no node. After: `node -> [10 | None]`. **Complexity:** $O(1)$ time and space. **Edge Cases:** Value may be any Python object. **Common Mistake:** Forgetting `self.next = None`; later pointer logic becomes unpredictable.

Interview Discussion

A node is not the stack; it is the building block of the stack.

1.7.5 Linked Method: `LinkedStack.__init__`

Problem

The stack needs to know where the top node is.

Need

An empty linked stack starts with no head and count zero.

Analogy

An empty chain has no first link.

Logic: Set `head=None`; set `count=0`. **Dry Run:**

Field	Value	Meaning
head	None	no top node
count	0	no values

Pseudo Code:

```
CREATE stack:
    head = None
    count = 0
```

Python Code and Driver:

```
1 stack = LinkedStack()
2 print(stack.head)
3 print(stack.size())
```

Console Output:

```
None
0
```

Output Explanation: No top node exists and the size is zero. **Memory Before/After:** `stack -> head: None, count: 0`. **Complexity:** $O(1)$. **Edge Cases:** None for normal construction. **Common Mistake:** Not maintaining a count; then `size()` may require traversal.

Interview Discussion

Keeping `count` makes `size()` $O(1)$ instead of $O(n)$.

1.7.6 Linked Method: `push`

Problem

We need to add a new top node without traversing the list.

Need

Insertion at head is immediate because the stack already stores `head`.

Analogy

Clip a new train coach at the front of the train.

Pointer Diagram:

```
Before: head -> [30] -> [20] -> [10] -> None
Push 40:
new_node -> [40]
new_node.next = head
head = new_node
After:  head -> [40] -> [30] -> [20] -> [10] -> None
```

Logic: Create node; point it to old head; move head to new node; increment count. **Dry Run:**

Step	Action	head value	count
1	create Node(40)	30	3
2	new.next=head	30	3
3	head=new	40	4

Pseudo Code:

```
PUSH(value):
    new_node = Node(value)
    new_node.next = head
    head = new_node
    count = count + 1
```

Python Code and Driver:

```
stack = LinkedStack()
stack.push(10)
stack.push(20)
stack.push(30)
stack.display()
```

Console Output:

```
Top
30
20
10
```

Output Explanation: Each new value becomes the head. Therefore `30` is the top, followed by the previous heads `20` and `10`. **Memory Before/After:** Before: `head -> [20] -> [10]`. After `push(30)`: `head -> [30] -> [20] -> [10]`. **Complexity:** $O(1)$ time, $O(1)$ extra space per node. **Edge Cases:** Empty stack works because `new_node.next` becomes `None`. **Common Mistake:** Setting `head = new_node` before `new_node.next = head`; that loses the old chain.

Interview Discussion

Linked-list stack push is $O(1)$ because it rewires a constant number of references.

1.7.7 Linked Method: `pop`

Problem

We need to remove the top node and make the next node the new top.

Need

This is the linked-list version of undoing the latest action.

Analogy

Detach the front coach; the next coach becomes first.

Pointer Diagram:

```
Before: head -> [30] -> [20] -> [10] -> None
removed_value = 30
head = head.next
After:  head -> [20] -> [10] -> None
```

Logic: Check empty; save head value; move head to next; decrement count; return saved value.
Dry Run:

Step	Action	head value	Return
1	is empty? no	30	–
2	save value	30	30
3	head=head.next	20	30

Pseudo Code:

```
POP():
    if empty: error
    value = head.value
    head = head.next
    count = count - 1
    return value
```

Python Code and Driver:

```
stack = LinkedStack()
for value in [10, 20, 30]:
    stack.push(value)
print(stack.pop())
stack.display()
```

Console Output:

```
30
Top
20
10
```

Output Explanation: The head node stored `30`, so it is removed and returned. The node storing `20` becomes the new head/top. **Memory Before/After:** Before: `head->[30]->[20]->[10]`. After: `head->[20]->[10]`; removed node is no longer reachable and can be garbage-collected. **Complexity:** $O(1)$ time and space. **Edge Cases:** Empty stack underflow; one-node stack becomes `head=None`. **Common Mistake:** Moving head first and then trying to read removed value; the old head may be lost.

Interview Discussion

In Python, unreachable nodes are cleaned by garbage collection; in lower-level languages you may need manual deallocation.

1.7.8 Linked Method: `peek`

Problem

We need to read the top node without detaching it.

Need

Peek lets us inspect state without changing pointer structure.

Analogy

Read the label on the front coach without uncoupling it.

Logic: If not empty, return `head.value`. **Dry Run:**

Step	Action	Chain	Return
1	read head.value	[30]->[20]	30
2	no mutation	[30]->[20]	30

Pseudo Code:

```
PEEK():
    if empty: error
    return head.value
```

Python Code and Driver:

```
stack = LinkedStack()
stack.push("first")
stack.push("second")
print(stack.peek())
print(stack.size())
```

Console Output:

```
second
2
```

Output Explanation: `second` is at head and remains there, so size is still 2. **Memory Before/After:** `head->[second]->[first]` remains unchanged. **Complexity:** $O(1)$. **Edge Cases:** Empty stack must be handled. **Common Mistake:** Returning `head.next.value`; that reads the second item, not top.

Interview Discussion

For a linked stack, top is `head`, not tail.

1.7.9 Linked Method: `is_empty`

Problem

We need a safe way to know whether `head.value` is valid.

Need

It prevents dereferencing `None`.

Analogy

Check whether the chain has a first link before pulling it.

Logic: Return `self.head is None`. **Dry Run:**

head	count	Result
None	0	True
Node(10)	1	False

Pseudo Code:

```
IS_EMPTY(): return head is None
```

Python Code and Driver:

```
stack = LinkedStack()
print(stack.is_empty())
stack.push(10)
print(stack.is_empty())
```

Console Output:

```
True
False
```

Output Explanation: Empty stack has no head. After push, head points to a node. **Memory Before/After:** `head=None`; then `head->[10]`. The method itself does not mutate memory. **Complexity:** $O(1)$. **Edge Cases:** If count and head disagree, the class invariant is broken. **Common Mistake:** Checking `count == None`; counts are numbers, not node references.

Interview Discussion

Using `head is None` is the clearest linked-list emptiness test.

1.7.10 Linked Method: `size` and `__len__`

Problem

Counting nodes by traversal every time can be slow.

Need

A maintained counter gives instant size.

Analogy

Instead of recounting plates every time, keep a tally updated when plates are added or removed.

Logic: Return `count`; update it in push and pop. **Dry Run:**

Operation	count before	count after
push	0	1
push	1	2
pop	2	1

Pseudo Code:

```
SIZE(): return count
__LEN__(): return SIZE()
```

Python Code and Driver:

```
stack = LinkedStack()
stack.push(10)
stack.push(20)
print(stack.size())
print(len(stack))
```

Console Output:

```
2
2
```

Output Explanation: The counter was incremented twice, so both calls return 2. **Memory Before/After:** Reading size does not alter `head->[20]->[10]`. **Complexity:** $O(1)$ with count; $O(n)$ if traversing. **Edge Cases:** Count must never become negative. **Common Mistake:** Forgetting to decrement count in pop.

Interview Discussion

A stored count is a deliberate space-time tradeoff: one integer for constant-time size.

1.7.11 Linked Method: `display` and `__str__`

Problem

Linked nodes are not stored contiguously, so printing requires traversal.

Need

Display helps verify pointer order.

Analogy

Walk along the chain from the first link to the last.

Logic: Start at head; print current value; move to current.next until None. **Dry Run:**

current	Print	Move to	Continue?
Node(30)	30	Node(20)	yes
Node(20)	20	Node(10)	yes
Node(10)	10	None	no

Pseudo Code:

```
DISPLAY():
    current = head
    while current is not None:
        print current.value
        current = current.next
```

Python Code and Driver:

```
stack = LinkedStack()
for value in [10, 20, 30]:
    stack.push(value)
stack.display()
print(str(stack))
```

Console Output:

```
Top
30
20
10
Top -> 30 -> 20 -> 10
```

Output Explanation: Traversal starts from head, and head is the top. Therefore output is already top-to-bottom. **Memory Before/After:** `head->[30]->[20]->[10]->None`; display does not change links. **Complexity:** $O(n)$ time; $O(1)$ extra space for display, $O(n)$ for string construction. **Edge Cases:** Empty stack prints only `Top`; string may become `Top ->` . **Common Mistake:** Forgetting `current = current.next`, causing an infinite loop.

Interview Discussion

Traversal is necessary for display, but not for push, pop, or peek.

1.8 Part 7: Array vs Linked List Stack

Aspect	Python List / Dynamic Array	Singly Linked List
Memory layout	Contiguous logical array slots	Nodes can live anywhere in memory
Top choice	End of list	Head node
Push	`append`, amortized $O(1)$	Insert at head, $O(1)$
Pop	End `pop`, $O(1)$	Remove head, $O(1)$
Peek	`items[-1]`	`head.value`
Cache locality	Better because nearby elements are close	Weaker because references jump between nodes
Extra memory	Less per element	Extra `next` reference per node
Dynamic size	Managed by Python list resizing	Naturally grows node by node
Implementation	Shorter and Pythonic	Teaches pointer manipulation
Interview value	Shows practical Python skill	Shows data-structure fundamentals
Best use	Most Python applications	When learning links or needing custom node behavior

1.9 Part 8: Underflow

Problem

What happens if you pop from an empty stack? There is no top item to remove.

Need

Underflow protection prevents invalid memory or invalid reference access.

```
stack = ListStack()
print(stack.pop())
```

```
IndexError: pop from empty stack
```

Professionals handle underflow by checking `is_empty()` or raising a clear exception. Returning `None` is possible, but dangerous if `None` is also a valid stack value.

1.10 Part 9: Overflow

Problem

A fixed-size stack may run out of reserved slots.

Need

Overflow tells us the structure cannot accept more items under its current capacity rule.

In a fixed-size array stack, `push` checks whether `top == capacity - 1`. In a Python list stack, capacity grows dynamically, so normal educational stacks do not expose `isFull`. Real systems can still fail from memory exhaustion.

```
Fixed capacity = 3
[10, 20, 30]
push(40) -> OverflowError
```

1.11 Part 10: Real-World Applications

Application	Why Stack Is Used
Browser Back Button	The most recently visited previous page is the first page to return to.
Undo / Redo	Latest action should be undone first; redo can use another stack.
Function Call Stack	Latest function call must finish before its caller continues.
Recursion	Each recursive call waits on top of previous calls.
Balanced Parentheses	Latest opening bracket must match the next closing bracket.
Expression Evaluation	Operators and operands are resolved using recent pending context.
Syntax Parsing	Compilers track nested structures with stack discipline.
Depth First Search	DFS remembers the latest discovered path before backtracking.
Backtracking	Latest decision is undone first when a path fails.
Maze Solver	Move history is stored so the solver can backtrack.
Compiler	Parsing, semantic scopes, and code generation often use stacks.
Virtual Machine	Operand stacks evaluate bytecode instructions.

Operating System	Threads use call stacks to manage active function frames.

1.12 Part 11: Python Function Call Stack

Problem

When one function calls another, Python must remember where to return and what local variables belong to each call.

Need

A stack is ideal because the newest function call must complete first.

Analogy

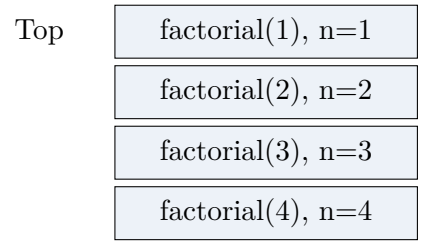
Nested boxes: you open box A, inside it box B, inside it box C. You must close C before B, and B before A.

```python
def factorial(n):
    if n == 1:
        return 1
    return n * factorial(n - 1)

print(factorial(4))
```

```
Call stack grows:
factorial(4)
factorial(3)
factorial(2)
factorial(1)

Call stack shrinks:
factorial(1) returns 1
factorial(2) returns 2
factorial(3) returns 6
factorial(4) returns 24
```



Each function call creates a **frame**: local variables, return address, and execution state. When a function returns, its frame is destroyed, and control returns to the frame below it.

1.13 Part 12: Interview Questions and Thinking

Interview Discussion

1. **Why is push $O(1)$?** It touches only the top end: append at list end or insert at linked-list head.
2. **Why is pop $O(1)$?** It removes only the top: list end or linked-list head.
3. **Why is peek $O(1)$?** It reads the top reference/index without traversal.
4. **Stack vs Queue?** Stack is LIFO; queue is FIFO.
5. **Array stack vs linked stack?** Array stack has better locality and simpler code; linked stack has explicit node growth and pointer practice.
6. **Can stack be implemented using queues?** Yes, by making either push or pop costly using one or two queues.
7. **Can queue be implemented using stacks?** Yes, commonly with two stacks: input stack and output stack.

1.14 Part 13: Forty Common Beginner Mistakes

#	Mistake	Why It Happens	Expert Fix
1	Calling `pop` on empty stack	Forgetting underflow	Check `is_empty` first
2	Calling `peek` on empty stack	Assuming top exists	Raise clear exception
3	Returning top in `pop` but not removing	Confusing peek and pop	Verify size decreases
4	Removing in `peek`	Using `pop()` accidentally	Verify size unchanged
5	Using `insert(0)` for list push	Thinking vertical top is index 0	Use end as top
6	Using `pop(0)`	Same visual confusion	Use `pop()`
7	Printing instead of returning	Mixing UI with logic	Methods should return values
8	Returning `None` silently on underflow	Avoiding exceptions	Document or raise error
9	Forgetting `self`	New class syntax confusion	Use instance fields
10	Shared class-level list	Misplacing `items=[]`	Initialize in constructor
11	Treating bottom as top	Misreading list order	Define invariant early
12	Not updating linked count on push	Counter forgotten	Update with mutation
13	Not updating linked count on pop	Counter forgotten	Test size after pop
14	Losing old head during linked push	Wrong assignment order	Set new.next before head change
15	Reading old head after moving it	Wrong pop order	Save value first
16	Infinite display loop	Forgetting current movement	Advance `current`
17	Traversing for linked push	Tail thinking	Use head as top

18	Traversing for linked pop	Tail thinking	Remove head
19	Not handling one-node pop	Edge case skipped	head becomes None
20	Using `== None` everywhere	Style habit	Prefer `is None`
21	Assuming list append is always worst-case $O(1)$	Ignoring resizing	Say amortized $O(1)$
22	Calling display a core ADT operation	Debugging confusion	Label as helper
23	Confusing stack with queue	LIFO/FIFO mixup	Use plate vs line analogy
24	Forgetting duplicate values are allowed	Set mental model	Stack stores sequence
25	Rejecting `None` unintentionally	Sentinel confusion	Decide API policy
26	Using global list	Avoiding classes	Encapsulate storage
27	Exposing internal list too freely	Convenience	Keep invariant protected
28	Comparing stack objects by print output	Testing shortcut	Test returned values
29	Not writing driver code	Memorizing methods	Exercise behavior
30	Skipping dry runs	Overconfidence	Trace top changes
31	Drawing linked arrows backward	Pointer confusion	Arrows follow `next`
32	Thinking linked list nodes are contiguous	Array mental model	Draw separate boxes
33	Thinking array stack uses pointers per element	Linked mental model	Use indices
34	Forgetting top after push	Not updating mental model	New item is top
35	Forgetting top after pop	Not updating mental model	Previous item becomes top
36	Returning size by traversal in list stack	Overgeneralizing linked lists	Use `len`
37	Not using exceptions consistently	API design gap	Choose one policy
38	Catching all exceptions blindly	Hiding bugs	Catch specific errors
39	Mutating while displaying	Debug code mixed with logic	Display should read only
40	Memorizing code without invariant	Surface learning	Repeat: top is only active end

1.15 Part 14: Practice Questions

Practice

Easy

1. Draw stack after push 5, 7, 9.
2. What does pop return?
3. What does peek return after two pushes?
4. Is an empty stack full?
5. Define LIFO in your own words.

Medium

1. Implement list stack without looking.
2. Implement linked stack push and explain pointer order.
3. Dry-run push 10, push 20, pop, push 30, peek.
4. Predict output of mixed display and pop calls.
5. Debug a pop method that returns but does not remove.

Hard

1. Implement balanced parentheses checker.
2. Implement queue using two stacks.
3. Implement stack using two queues.
4. Build a min-stack supporting `get_min()` in $O(1)$.
5. Trace recursive factorial using call-stack frames.

1.16 Part 15: Mini Projects

1.16.1 Undo System

Use one stack to store actions. Push each edit. Pop to undo latest edit.

1.16.2 Browser History

Push visited pages. Back pops current page and reveals previous page. A second stack can support Forward.

1.16.3 Balanced Parentheses Checker

Push opening brackets. On closing bracket, pop and match. Empty at end means balanced.

1.16.4 Reverse String

Push each character, then pop until empty.

1.16.5 Decimal to Binary Converter

Repeatedly push remainders from division by 2; pop to read binary digits in correct order.

1.16.6 Expression Evaluator

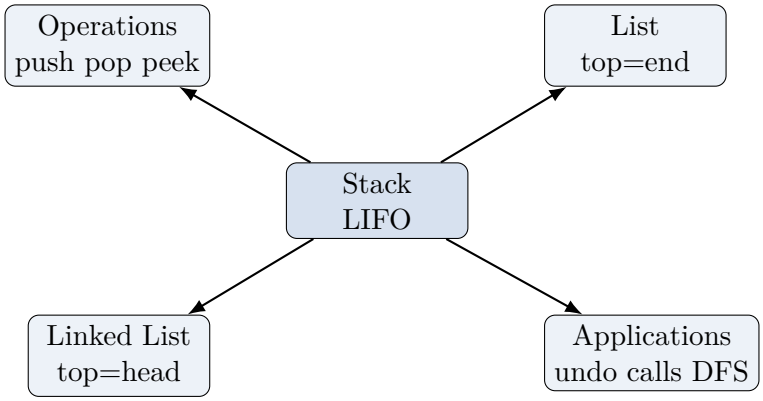
Use stacks for operands and operators, or evaluate postfix expressions with one operand stack.

1.17 Part 16: Summary

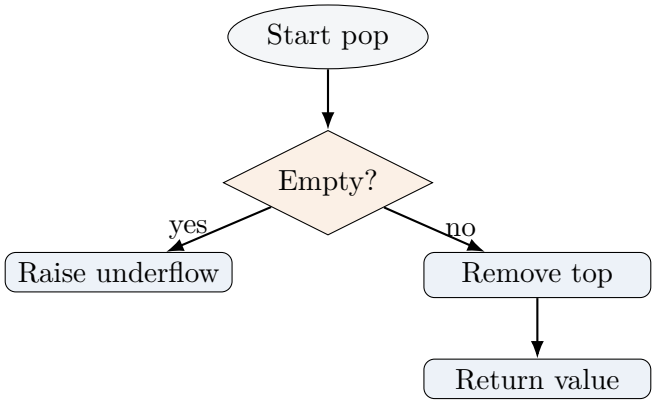
1.17.1 Cheat Sheet

Concept	List Stack	Linked Stack
Top	last index	head
Push	`append(value)`	new node before head
Pop	`pop()`	remove head
Peek	`items[-1]`	`head.value`
Empty	`len(items)==0`	`head is None`
Display	reversed list	traverse from head

1.17.2 Mind Map



1.17.3 Flow Chart for Pop



1.17.4 Operation Summary and Complexity Table

Operation	List Stack	Linked Stack
push	amortized $O(1)$	$O(1)$
pop	$O(1)$	$O(1)$
peek	$O(1)$	$O(1)$
is_empty	$O(1)$	$O(1)$
size	$O(1)$	$O(1)$ with count
display	$O(n)$	$O(n)$

1.17.5 Memory Tricks

- List stack: **right end is top**; draw indices left to right, display top to bottom.
- Linked stack: **head is top**; push and pop change only the first node.
- Pop changes memory; peek does not.
- Underflow means no top exists.
- Overflow matters for fixed-size stacks; Python lists grow dynamically until memory limits.

Expert Tip

Final invariant to remember: a stack has exactly one active end. If your operation touches the bottom or traverses unnecessarily for push/pop, pause and redesign.